

Auditoria de Arquitetura

Navix Route Intelligence

Revisor: Principal Architect / CTO · Data: 2026-07-12 · Escopo: monorepo completo (API, Web, contracts, infra, docs) · Modo: somente leitura — nenhum código alterado

Sumário executivo

A Navix é um projeto **acima da média em disciplina arquitetural**. A separação em camadas (Clean Architecture) é real e *automatizada por lint*, o DDD tem agregados com invariantes de verdade, e o isolamento multi-tenant usa **defesa em profundidade** (RLS FORCE + role sem privilégio + filtro explícito + transação por request). Poucos projetos nesta fase têm essa base.

O **risco central não é o código — é a distância entre a documentação e a implementação**. Vários ADRs marcados como “Aceito” (Outbox, eventos de domínio, CQRS/read models, criptografia envelope por tenant, fila assíncrona, TimescaleDB, Redis) **não existem em código**. Decisões marcadas como aceitas induzem stakeholders e novos devs a construir sobre garantias que não estão lá — por exemplo, assumir que PII está criptografada ou que eventos são entregues de forma confiável.

Veredito: fundação sólida e coerente, pronta para MVP. **Não** pronta para as metas de escala/produção que a documentação afirma, e a documentação precisa ser rebaixada para refletir a realidade.

Dimensão	Nota	Comentário
Arquitetura geral	Forte	Clean Arch coerente e aplicada
Organização de módulos	Forte	9 módulos com estrutura idêntica
Clean Architecture	Forte	Enforced por eslint-plugin-boundaries
DDD	Forte	Agregados, Value Objects, máquina de estados
SOLID	Forte	DIP via ports; baixo acoplamento
Multi-tenant	Forte	RLS FORCE + role runtime + interceptor
Segurança	Ressalvas	Backend forte; token no localStorage; cripto ausente
Escalabilidade	Risco	Rate-limit in-memory, mídia no OLTP, async inexistente
Performance	Bom	Índices e paginação OK; mídia base64 preocupa
Consistência entre módulos	Forte	Padrão muito uniforme
APIs	Forte	Versionadas, validadas, Swagger, envelope padrão
Banco de dados	Forte	Migrações limpas, RLS, índices por tenant
Docker	Médio	Funciona; falta non-root, healthcheck, npm ci
CI/CD	Risco	Não roda e2e/web tests; sem audit/scan; sem gate
Testes	Médio	Domínio bem coberto; controllers/repos fracos

Documentação	Ressalvas	Excelente forma; descreve o que não existe
--------------	------------------	--

Pontos fortes (preservar)

- **Fronteiras de módulo reais e automatizadas.** apps/api/.eslintrc.cjs configura eslint-plugin-boundaries para impedir domain → application/infra, etc. É o que mantém o “monólito modular” extraível.
- **Comunicação entre contextos por anti-corruption gateway.** O Optimizer consome a port pública DeliveryLookupPort, nunca tabelas alheias. Mesmo padrão em Import→Delivery, POD→Delivery, Delivery→Fleet.
- **Multi-tenant com defesa em profundidade genuína:** runtime conecta como role não-owner navix_app (NOBYPASSRLS); FORCE ROW LEVEL SECURITY + WITH CHECK em todas as tabelas de negócio; SET LOCAL app.current_tenant por transação via AsyncLocalStorage; e filtro explícito tenant_id nos repositórios. Há inclusive um tenant-isolation.e2e-spec.ts.
- **Segurança de autenticação bem feita:** JWT RS256 com key ring + rotação (kid), argon2id, refresh token opaco e hashado com rotação por family_id, helmet, CORS restrito, ValidationPipe com whitelist + forbidNonWhitelisted, filtro de exceções que não vaza internals, config validada por Zod (fail-fast), .env corretamente ignorado.
- **DDD de verdade:** Delivery é um aggregate root com construtor privado, factories create/restore, máquina de estados (canTransition) e Value Objects (Address, TimeWindow).
- **API consistente:** versionamento URI (/api/v1), paginação com @Max(100) em todos os DTOs de lista, envelope de resposta padronizado, Swagger só fora de produção.

CRÍTICO

C1. ADRs “Aceitos” descrevem arquitetura inexistente (risco de decisão)

Documentação e realidade divergem em pontos estruturais marcados como decididos:

ADR	Detalhamento
ADR-0006 Transactional Outbox	Tabela outbox existe; nenhum producer, relay ou consumer. Nenhum evento é escrito.
ADR-0007 Otimizador assíncrono via fila (202)	Otimização é síncrona (201), in-process — optimizer.controller.ts:45.
ADR-0011 CQRS / read models	Inexistente (zero materialized views / réplicas).
ADR-0002 Redis	ioredis declarado, zero imports; throttler é in-memory.
ADR-0009 TimescaleDB	Tabela comum; só um comentário mencionando hypertable.

Por que é crítico: não é dívida de código, é dívida de verdade. Um ADR “Aceito” é um contrato de time. Alguém vai assumir que “eventos são confiáveis” ou “relatórios não competem com o OLTP” e construir sobre o vazio. **Ação:** reclassificar esses ADRs para Proposto/Planejado e adicionar coluna “Implementado: sim/não/parcial” na tabela de decisões. Custo ~1h, elimina o maior risco do projeto.

C2. Criptografia de PII (ADR-0010) não existe, mas é anunciada como aceita

ENCRYPTION_KEK é validada no env.schema e nunca usada (nenhum createCipher/aes-256/envelope no código). Campos com PII — phone, display_name, endereços completos, photo/signature de comprovante — estão em texto puro. Se algum stakeholder (ou cláusula de LGPD/contrato) acredita que “envelope encryption por tenant” está ativa, há exposição de compliance. **Ação:** implementar cifragem de campo para PII sensível, ou rebaixar o ADR e registrar explicitamente que os dados estão em claro.

ALTO

A1. Rate limiting in-memory — inútil com múltiplas instâncias

throttling.module.ts usa storage em memória (o próprio comentário reconhece). Com 2+ réplicas atrás de LB, o limite efetivo multiplica pelo nº de instâncias e a proteção de brute-force no /login é contornável. Contradiz a meta de “milhares de tenants”. **Ação:** storage Redis (@nest-lab/throttler-storage-redis) — o Redis já está no compose e ocioso.

A2. Refresh token em localStorage (superfície de XSS)

auth-provider.tsx:54 persiste o refresh token (TTL 14 dias) em localStorage. Qualquer script injetado o exfiltra → sequestro de sessão de longa duração. É o elo fraco frente a um backend rigoroso (RS256/argon2/RLS). O access token em useRef (memória) está correto. **Ação:** mover o refresh token para cookie httpOnly + Secure + SameSite, emitido pela API.

A3. CI não roda os testes que existem

ci.yml roda npm test, que resolve para o jest unit da API apenas. Não executa os e2e (delivery, optimizer, tenant-isolation — inclusive o teste que valida o isolamento multi-tenant!) nem o vitest do web. Sem gate de cobertura apesar da meta declarada de $\geq 80\%$ em domínio/aplicação. **Ação:** adicionar steps de e2e (com Postgres já disponível no job) e de test do web; adicionar --coverage com threshold.

A4. Escalabilidade: mídia binária (base64) no OLTP

POD photo/signature e avatar são colunas text guardando data URLs base64 (até 8 MB de body na API). Isso incha o Postgres, degrada backups/replicação, estoura cache de páginas e briga com o read-path do OLTP — o oposto da meta de “milhões de entregas/dia”. **Ação:** object storage (S3/GCS) + URL assinada; guardar só a referência no banco.

MÉDIO

M1. Boundaries enforça só camadas, não isolamento entre módulos

A regra “inviolável” da doc (§4: um módulo nunca importa internals de outro) não está sendo aplicada. A config de boundaries/element-types classifica por camada (domain/application/...), então application de um módulo pode importar application de outro sem erro de lint. Hoje a disciplina é mantida à mão (via gateways). **Ação:** capturar o módulo como elemento e proibir cross-module fora de shared/contracts.

M2. Dependência xlsx@0.18.5 com CVEs conhecidas

A versão npm do SheetJS (0.18.5) tem prototype pollution e ReDoS conhecidos (correções só no registro próprio da SheetJS, ≥0.20). Como o Import Center faz parse de planilhas enviadas por usuários, é superfície de ataque real. **Ação:** migrar para o registro oficial da SheetJS ou avaliar exceljs. Adicionar npm audit/Dependabot ao CI.

M3. Dockerfiles: hardening e reprodutibilidade

api.Dockerfile / web.Dockerfile: rodam como root (sem USER node), usam npm install (não npm ci → build não reprodutível, ignora o lockfile), sem HEALTHCHECK, e o web usa npm run start em vez do output: 'standalone' do Next (imagem maior). Sem .dockerignore visível. **Ação:** npm ci, usuário não-root, healthcheck, standalone.

M4. CI mínimo, sem segurança nem build de imagem

Um único job, sem matrix de Node, sem npm audit/SAST/secret-scan, sem build/push das imagens Docker, sem cache de build. É “CI” mas não “CD”. **Ação:** adicionar etapa de segurança e (quando houver ambiente) build+publish de imagem.

M5. Redis provisionado e ocioso

Está no docker-compose, no env.schema e em package.json (ioredis), sem um único uso. Ou remover (reduz superfície e confusão) ou ativá-lo para o uso pretendido (throttler/cache/filas). Ver A1.

BAIXO

- **B1. README desatualizado:** afirma “Fase 0... frota/entregas/otimização ainda não implementadas”, mas existem 9 módulos funcionais. Atualizar o “Status”.
- **B2. tenancy/README.md obsoleto:** seção “A implementar” descreve o interceptor de tenant como pendente — ele já existe e funciona.
- **B3. Inconsistência de modelagem:** user-profile vive dentro de identity, mas user-settings é um módulo NestJS próprio — ambos são atributos 1:1 de users. Escolher um padrão (sugiro ambos dentro de identity).
- **B4. CreateAppRole interpola senha/role em DDL por string.** Origem é env (baixo risco), mas é um padrão a evitar; documentar que DB_APP_USER/PASSWORD são confiáveis.
- **B5. Sem observabilidade distribuída real:** logs estruturados (pino) OK, mas tracing/métricas da §10 da arquitetura não existem. Alinhar doc ↔ realidade.

Roadmap priorizado (ordem sugerida)

Item	Atualizações
1. Hoje (baixo custo, alto valor)	C1 + C2 — reconciliar ADRs/README/tenancy README com a realidade (coluna "Implementado"). Elimina o risco de decisão sem tocar em código.
2. Antes de qualquer piloto com dados reais	A2 (refresh token em cookie httpOnly), A1 (rate-limit no Redis), A3 (rodar e2e/web no CI — em especial o teste de isolamento de tenant).
3. Antes de escalar / vender escala	A4 (mídia em object storage), M1 (isolamento de módulo no lint), M2 (xlsx/CVE + npm audit no CI).
4. Endurecimento de plataforma	M3 (Docker non-root/npm ci/healthcheck), M4 (CI de segurança + imagens), M5 (decidir sobre Redis).
5. Contínuo	Subir cobertura de controllers/repositories; implementar de fato (ou remover da doc) outbox/eventos/CQRS/cripto conforme a fase exigir.